



Wyższa Szkoła Przedsiębiorczości i Administracji w Lublinie

Zaawansowany projekt zespołowy cz. 2

Autorzy: **Piotr Trochimiuk (kierownik), Sebastian Pidek, Jan Płoskonka, Mateusz Sochacki**

Prowadzący: **dr inż. Moniuszko**

Lublin 2026

Spis treści

1. Informacje Ogólne	3
2. Specyfikacja Techniczna	3
3. Architektura Systemu	3
4. Kluczowe Funkcjonalności	4
5. Przegląd Interfejsu (UX)	5
6. Bezpieczeństwo i Przechowywanie	5

1. Informacje Ogólne

Projekt **KasaMate** zakłada stworzenie nowoczesnej aplikacji mobilnej przeznaczonej do zarządzania finansami osobistymi użytkowników. Głównym celem systemu jest wsparcie użytkownika w codziennej kontroli przychodów i wydatków oraz zwiększenie świadomości finansowej poprzez przejrzystą prezentację danych i narzędzia analityczne.

Aplikacja umożliwia rejestrowanie wszystkich operacji finansowych, takich jak przychody oraz wydatki, wraz z przypisaniem ich do odpowiednich kategorii. Dzięki temu użytkownik może w łatwy sposób monitorować strukturę swoich wydatków oraz analizować, na jakie cele przeznaczane są poszczególne środki finansowe. System pozwala również na definiowanie własnych kategorii, co umożliwia dopasowanie aplikacji do indywidualnych potrzeb i stylu życia użytkownika.

2. Specyfikacja Techniczna

- Język: Kotlin 1.9+
- Framework UI: Jetpack Compose (Material Design 3)
- Architektura: MVVM (Model-View-ViewModel)
- Baza Danych: Room Persistence Library (SQLite)
- Zarządzanie Stanem: StateFlow, LiveData, observeAsState()
- Wykresy: MPAndroidChart (PhilJay)
- Nawigacja: Jetpack Navigation Compose

3. Architektura Systemu

3.1. Warstwa Prezentacji (UI)

Głównym komponentem interfejsu jest TransactionScreen, który odpowiada za:

- Wyświetlanie listy transakcji przy użyciu LazyColumn.
- Zarządzanie filtrowaniem danych na podstawie wybranego miesiąca i roku.
- Wyświetlanie podsumowań finansowych (całkowite wydatki, pozostały budżet).

3.2. Warstwa Logiki (ViewModel)

TransactionViewModel pośredniczy między bazą danych a widokiem. Wykorzystuje TransactionViewModelFactory, aby dostarczyć dane specyficzne dla zalogowanego użytkownika (username).

3.3. Warstwa Danych

- Transaction: Encja Room reprezentująca pojedynczy wpis (opis, kwota, data, typ: przychód/wydatek).
- Settingsrepository: Zarządza ustawieniami użytkownika (np. miesięczny budżet) przy użyciu SharedPreferences.

4. Kluczowe Funkcjonalności

4.1. Zarządzanie Transakcjami

- Użytkownik ma możliwość pełnej kontroli nad swoimi operacjami finansowymi:
- Dodawanie wydatków: Przycisk "Dodaj transakcję" otwiera dialog AddTransactionDialog.
- Dodawanie przychodów: Funkcja AddIncomeDialog pozwala zwiększyć saldo i budżet.
- Edycja/Usuwanie: Kliknięcie w element listy pozwala na modyfikację istniejącego wpisu.
- Masowe usuwanie: Ikona kosza w TopBar pozwala na wyczyszczenie całej bazy danych dla danego użytkownika.

4.2. Zaawansowane Filtrowanie i Sortowanie

- Selektor Daty: Aplikacja posiada wbudowany MonthYearPickerDialog oraz nawigację strzałkami, pozwalającą przeglądać dane z konkretnych miesięcy.
- Sortowanie: Dostępne opcje sortowania (poprzez SortType):
- Nazwa (A-Z / Z-A)
- Cena (Rosnąco / Malejąco)
- Data (Najnowsze / Najstarsze)

4.3. Monitoring Budżetu

- System dynamicznie oblicza stan finansów:
- Total Spent: Suma wszystkich transakcji niebędących przychodami w wybranym miesiącu.
- Remaining Budget: Różnica między budżetem ustawionym a wydatkami. Jeśli kwota jest ujemna, interfejs zmienia kolor na MaterialTheme.colorScheme.error (czerwony).

4.4. Wizualizacja Danych

Komponent MonthlySummaryChart integruje bibliotekę MPAndroidChart z Compose, wyświetlając graficzne podsumowanie aktywności finansowej użytkownika.

5. Przegląd Interfejsu (UX)

- Scaffold: Zapewnia standardowy układ aplikacji z paskiem górnym (AppBar) i obszarem treści.
- DropdownMenu: Używane do nawigacji (Ustawienia, O programie, Wyloguj) oraz wyboru opcji sortowania.
- Responsywność: Zastosowanie `Modifier.weight()` zapewnia poprawne wyświetlanie przycisków akcji na ekranach o różnej szerokości.

6. Bezpieczeństwo i Przechowywanie

- Dane są izolowane na poziomie bazy danych poprzez klucz `username`.
- Wymagany jest proces logowania (`LoginScreen`), który inicjuje `TransactionViewModel` z odpowiednim kontekstem użytkownika.